React – Props, State, Events

Rappel Les propriétés (Props)

Définition

- Une prop est une donnée transmise du parent vers l'enfant.
- Elles permettent de rendre les composants réutilisables avec des valeurs différentes.
- Les props sont en lecture seule pour le composant enfant.

1. Exemples avec différents types de props

a) String (texte)

```
function Message(props) {
  return <p>Message : {props.texte}</p>;
}

function App() {
  return <Message texte="Bonjour à tous !" />;
}
```

1. **function Message(props)** : props est un objet contenant toutes les props passées par le parent.
2. **{props.texte}** : récupère la valeur de la prop texte ("Bonjour à tous !").
3. **<Message texte="Bonjour à tous !" />** : le parent crée une instance de Message avec une prop texte.

b) Number (nombre)

```
function AfficheAge(props) {
  return <p>Âge : {props.age} ans</p>;
}
function App() {
  return <AfficheAge age={25} />;
}
```

1. **props.age récupère la valeur 25 passée par le parent.**
2. **Cela permet de réutiliser AfficheAge pour plusieurs âges différents sans modifier le composant.**

c) Boolean (booléen)

```
function Status(props) {
  return <p>Est connecté ? {props.isConnected ? "Oui" : "Non"}</p>;
}
function App() {
  return <Status isConnected={true} />;
}
```

1. **props.isConnected est vrai, donc le texte affiché est "Oui".**
2. **Si isConnected={false}, l'enfant afficherait "Non".**

d) Tableau

```
function Liste(props) {
  return (
    <ul>
      {props.items.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>
  );
}
```

```
function App() {
  const fruits = ["Pomme", "Banane", "Orange"];
  return <Liste items={fruits} />;
}
```

1. **props.items contient le tableau ["Pomme", "Banane", "Orange"].**
2. **map crée un <li> pour chaque élément.**
3. **Chaque élément a un key unique pour que React gère bien le DOM.**

Définition de map()

La méthode **map()** permet de **parcourir un tableau** et de **créer un nouveau tableau** en appliquant une fonction à **chaque élément** du tableau d'origine.

Explication :

- tu as une liste de fruits : ["Pomme", "Banane", "Orange"]
- Tu veux faire **quelque chose pour chaque fruit**, par exemple l'afficher à l'écran.
- Au lieu d'utiliser une boucle for, on peut utiliser **map()**.

Exemple :

```
const fruits = ["Pomme", "Banane", "Orange"];
const resultat = fruits.map((fruit) => "J’aime " + fruit);
console.log(resultat);
```

Résultat dans la console :

["J’aime Pomme", "J’aime Banane", "J’aime Orange"]

Dans ton l’exemple props d) Tableau

```
{props.items.ap((item, index) => (
  <li key={index}>{item}</li>
))}
```

Ici, map() sert à :

- parcourir le tableau props.items
- et **créer un <li> pour chaque élément** du tableau

e) Objet

```
function Profil(props) {
  return (
    <div>
      <h2>{props.user.nom}</h2>
      <p>Âge : {props.user.age}</p>
    </div>
  );
}
```

```
function App() {
  const user = { nom: "Ali", age: 25 };
  return <Profil user={user} />;
}
```

1. **props.user est un objet { nom, age }.**
2. **L'enfant peut accéder aux différentes propriétés comme props.user.nom et props.user.age.**

f) Fonction

```
function Bouton(props) {
  return <button onClick={props.onClick}>Cliquez-moi</button>;
}
function App() {
  const handleClick = () => alert("Bonjour !");
  return <Bouton onClick={handleClick} />;
}
```

- **props.onClick est une fonction passée par le parent.**
- **Quand l'enfant clique sur le bouton, la fonction handleClick du parent est exécutée.**

g) Composant comme prop

```
function Wrapper(props) {
  return (
    <div style={{ border: "2px solid black", padding: "10px" }}>
      {props.enfant}
    </div>
  );
}

function App() {
  return <Wrapper enfant={<p>Je suis un composant enfant passé en prop !</p>} />;
}
```

1. **props.enfant contient un composant JSX.**
2. **Le composant Wrapper affiche ce JSX à l'endroit où {props.enfant} est utilisé.**

4. Exemple combinant plusieurs types

```
function ProfilComplet(props) {
  return (
    <div>
      <h2>{props.nom}</h2>
      <p>Âge : {props.age}</p>
      <p>Admin : {props.isAdmin ? "Oui" : "Non"}</p>
      <p>Compétences :</p>
      <ul>
        {props.skills.map((skill, index) => (
          <li key={index}>{skill}</li>
        ))}
      </ul>
      <button onClick={props.onClick}>Envoyer un message</button>
    </div>
  );
}
function App() {
  const skills = ["HTML", "CSS", "JavaScript"];
```

```
  const sayHello = () => alert("Salut !");
  return (
    <ProfilComplet
      nom="Ali"
      age={25}
      isAdmin={true}
      skills={skills}
      onClick={sayHello}
    />
  );
}
```

Explication :

Ce code montre comment un composant React (**ProfilComplet**) peut recevoir **plusieurs types de props en même temps** :

- des chaînes de caractères (string),
- des nombres,
- des booléens,
- des tableaux,
- et même une fonction.

1) Définition du composant enfant ProfilComplet

- function ProfilComplet(props) { ... }
  → **Déclare** un composant fonctionnel nommé ProfilComplet.
  → props est le **paramètre d'entrée** : un objet qui contiendra toutes les propriétés envoyées par le parent.

2) Début du JSX retourné par ProfilComplet

- return (...);
  → Le composant renvoie un bloc JSX qui sera transformé par React en éléments DOM. Tout ce qui est entre ... sera affiché quand le composant est rendu.

3) Affichage du nom

- <h2>{props.nom}</h2>
  → **Récupère** props.nom (la prop nom passée par le parent) et l'affiche dans un titre <h2>.
  → Si props.nom === "Ali", on voit "Ali".

4) Affichage de l'âge

- Âge : {props.age}
  → Lit props.age (nombre) et concatène dans le paragraphe.
  → Résultat exemple : "Âge : 25".

5) Condition ternaire pour le statut admin

- Admin : {props.isAdmin ? "Oui" : "Non"}
  → Évalue props.isAdmin (booléen).
  - Si true → affiche "Oui".

- Si false → affiche "Non".
  → Ici isAdmin={true} donc affichera “Admin : Oui”.

6) Titre de la section compétences

- Compétences :
  → Simple texte statique pour introduire la liste.

7) Début de la liste <ul> et itération du tableau skills

- <ul> {props.skills.map((skill, index) => (<li key={index}>{skill}</li>))} </ul>
  Décomposons :
  - props.skills : **doit être un tableau** (ici ["HTML","CSS","JavaScript"]).
  - .map((skill, index) => ...) : parcourt chaque élément du tableau et crée un <li> pour chacun.
    - skill prend successivement "HTML", "CSS", "JavaScript".
    - index est la position (0, 1, 2).
  - Pour chaque itération React crée un <li key={index}>{skill}</li>.
    - key={index} : **clé unique** pour chaque <li>. Aide React à optimiser les mises à jour du DOM. (Remarque pédagogique : pour des listes dynamiques il vaut mieux une clé stable provenant des données plutôt que index.)

- Résultat rendu :
- • HTML
- • CSS
- • JavaScript

8) Bouton et gestionnaire d'événement

- <button onClick={props.onClick}>Envoyer un message</button>
  → Le bouton reçoit un **handler** onClick qui **référence** props.onClick (une fonction fournie par le parent).
  → IMPORTANT : on **passe la référence** à la fonction (sans les parenthèses) — React l'appellera au clic.
  → Quand l'utilisateur clique, React exécute props.onClick().

9) Fin du composant ProfilComplet

- } ferme la fonction. Le composant est prêt à être utilisé.

10) Définition du composant parent App

- function App() { ... }
  → Déclare le composant parent.

11) Déclaration de la donnée skills

- const skills = ["HTML", "CSS", "JavaScript"];
  → **Crée** un tableau nommé skills contenant trois chaînes.

→ C’est la donnée qui sera passée au composant enfant via la prop skills.

12) Déclaration de la fonction sayHello

- const sayHello = () => alert("Salut !");
  → **Crée** une fonction fléchée qui déclenche une alerte navigateur avec le texte "Salut !".
  → Cette fonction sera passée comme prop onClick au composant enfant.

13) Rendu du composant ProfilComplet par App

- Le return de App contient :

```
<ProfilComplet
  nom="Ali"
  age={25}
  isAdmin={true}
  skills={skills}
  onClick={sayHello}
/>
```

→ **Instancie** ProfilComplet et **envoie** ces props :

- nom : chaîne "Ali"
- age : nombre 25 (passé entre {})
- isAdmin : booléen true

- skills : le **tableau** skills défini avant
- onClick : la **fonction** sayHello

14) Montage et rendu final

- React exécute App → crée l'arbre JSX → convertit en DOM → monte ProfilComplet dans le DOM racine.
- L'utilisateur voit la structure HTML décrite plus haut (titre, âge, statut, liste, bouton).

15) Interaction utilisateur (clic sur le bouton)

- Quand l'utilisateur clique sur **Envoyer un message** :
  1. React détecte l'événement click.
  2. Il appelle la fonction référencée par onClick, c'est-à-dire props.onClick.
  3. props.onClick pointe vers sayHello (déclarée dans App), donc sayHello() s'exécute.
  4. sayHello() appelle alert("Salut !") → la boîte d'alerte apparaît.

2. Utilisation des props dans les classes React

Définition

- Dans une **classe** React, les props sont accessibles via this.props.
- Comme dans les fonctions, elles sont en lecture seule.

Exemple

```
import React, { Component } from "react";
class Message extends Component {
  render() {
    return <p>Bonjour {this.props.nom} !</p>;
  }
}
class App extends Component {
  render() {
    return (
      <div>
        <Message nom="Ali" />
        <Message nom="Sara" />
      </div>
```

```
    );
  }
}
```

Explication :

- **this.props.nom : permet d'accéder à la valeur passée par le parent.**
- **Chaque instance reçoit ses propres props.**
- **Utile pour gérer des composants plus complexes avec méthodes et state.**

La gestion d'état (States) en React

Qu'est-ce que le "state" ?

- Le **state** (ou **état**) est une donnée interne au composant.
- Il permet à un composant de mémoriser des informations qui peuvent changer pendant l'exécution de l'application.
- Quand la valeur du state change, React met automatiquement à jour l'affichage du composant dans la page.

Le state sert à rendre les composants dynamiques, capables de réagir aux actions de l'utilisateur (clic, saisie, etc.).

Différence entre Props et State

Aspect	Props	State
Origine	Données reçues du composant parent	Données internes du composant
Modification	Immuables (lecture seule)	Modifiables par le composant lui-même
Utilisation	Transmettre des valeurs	Stocker des informations qui évoluent
Exemple	nom="Ali" (fixe)	count=0 → 1 → 2 (variable)

- **Les props sont données *de l'extérieur*.**
- **Le state appartient au composant et *peut changer avec le temps*.**

Définir un state dans un composant (classe)

Avant les **hooks**, le state se déclare uniquement dans une classe.

Exemple : **compteur**

```
import React, { Component } from "react";
class Compteur extends Component {
  //Constructeur : sert à initialiser le state
  constructor(props) {
```

```
  super(props); // Appel du constructeur de la classe parente (React.Component)
  // Déclaration du state avec une valeur initiale
  this.state = { count: 0 };
 }
 // Méthode pour modifier le state
 incrementer = () => {
  // setState() met à jour le state et relance le rendu
  this.setState({ count: this.state.count + 1 });
 };
 // Méthode d'affichage (obligatoire)
 render() {
  return (
   <div>
    <h2>Compteur : {this.state.count}</h2>
    <button onClick={this.incrementer}>+</button>
   </div>
  );
 }
}
```

export default Compteur;

Explication :

➤ 1. this.state = { count: 0 };

- On déclare un objet d'état qui contient une variable count.
- Sa valeur initiale est 0.
- Ce state appartient uniquement à ce composant.

➤ 2. this.setState({ count: this.state.count + 1 });

- Permet de modifier la valeur du state.
- React détecte le changement et rafraîchit automatiquement l'affichage du composant.
- Important : on ne modifie jamais this.state directement.
  this.state.count = 1
  this.setState({ count: 1 })

➤ 3. render()

- C'est la méthode obligatoire qui décrit ce que le composant doit afficher à l'écran.
- Chaque fois que le state change, React rappelle render() pour mettre à jour l'affichage.

➤ 4. <button onClick={this.incrementer}>+</button>

- Quand l'utilisateur clique, la méthode incrementer() est appelée.
- Elle modifie le state grâce à setState() → React met à jour la valeur affichée.

Exemple 2 : Compteur avec deux boutons (incrémenter / décrémenter)

```
import React, { Component } from "react";

class Compteur2 extends Component {

  constructor(props) {

    super(props);

    this.state = { count: 0 };

  }


  incrementer = () => {

    this.setState({ count: this.state.count + 1 });

  };


  decrementer = () => {

    this.setState({ count: this.state.count - 1 });

  };
```

```
  render() {
    return (
      <div>
        <h2>Valeur du compteur : {this.state.count}</h2>
        <button onClick={this.incrementer}>+</button>
        <button onClick={this.decrementer}>-</button>
      </div>
    );
  }
}
export default Compteur2;
```

Ici :

- **Le bouton + augmente la valeur.**
- **Le bouton - la diminue.**
- **Le composant se met à jour automatiquement à chaque clic.**

Exemple 3 : Changement d'état texte

Le state ne contient pas que des nombres. Il peut aussi stocker du texte, des booléens, des tableaux, etc.

```
import React, { Component } from "react";
class Message extends Component {
  constructor(props) {
    super(props);
    this.state = { message: "Bonjour !" };
  }

  changerMessage = () => {
    this.setState({ message: "Bienvenue sur React !" });
  };

  render() {
    return (
      <div>
        <p>{this.state.message}</p>
        <button onClick={this.changerMessage}>Changer le
message</button>
      </div>
    );
  }
```

```
}

export default Message;
```

Résultat :

- Au départ : “Bonjour !”
- Après clic sur le bouton : “Bienvenue sur React !”

Règles importantes sur le State

Toujours utiliser **setState**() pour modifier la valeur.
Le state doit être initialisé dans le constructeur.
Chaque changement du state provoque un nouveau rendu du composant.

Le state peut contenir plusieurs propriétés :

```
this.state = {
  nom: "Ali",
  age: 25,
  ville: "Agadir"
};
```

et plus tard :

```
this.setState({ age: 26 });
```

Résumé général

Concept	Description	Exemple
State	Données internes d'un composant	this.state = { count: 0 }
Modification	Via this.setState()	this.setState({ count: 1 })
Réaction	Chaque modification déclenche un nouveau rendu	compteur qui s'actualise
Props vs State	Props = données reçues ; State = données locales	nom vs count

À retenir

Le state, c'est la mémoire vivante du composant.
Il permet à un composant d'évoluer sans recharger la page.
Il rend ton interface interactive et dynamique.

Gestion des Événements et Communication entre Composants (Version classes)

Qu'est-ce qu'un événement ?

Un **événement** est une action effectuée par l'utilisateur :

- cliquer sur un bouton
- taper dans un champ
- soumettre un formulaire
- passer la souris, etc.

En React, on réagit à ces actions avec des fonctions appelées gestionnaires d'événements (event handlers).

Différence entre HTML et React

En HTML classique	En React (JSX)
Les événements sont en minuscules : onclick, onchange	En camelCase : onClick, onChange
On met la fonction entre guillemets	On passe une référence à la fonction entre {}

Exemple :

HTML

```
<button onclick="afficherMessage()">Cliquez-moi</button>
```

React

```
<button onClick={this.afficherMessage}>Cliquez-moi</button>
```

Les événements courants

onClick

```
import React, { Component } from "react";
class Bouton extends Component {
  afficherMessage() {
    alert("Vous avez cliqué !");
  }

  render() {
    return <button onClick={this.afficherMessage}>Cliquez-moi</button>;
  }
}
export default Bouton;
```

Ici, this n'est pas automatiquement lié à la classe.
On peut utiliser une fonction fléchée ou .bind(this)

onSubmit

```
// On importe React et la classe "Component" depuis la bibliothèque "react"
import React, { Component } from "react";
// On crée une nouvelle classe appelée "Formulaire" qui hérite de la classe "Component" de React
class Formulaire extends Component {
 // Le constructeur est une fonction spéciale appelée automatiquement
 // quand on crée une nouvelle instance (un nouvel objet) de la classe
  constructor() {
    super();// On appelle le constructeur de la classe "Component" (obligatoire)
// On lie la méthode "handleSubmit" à l'objet courant (this)
   // Cela permet d'utiliser "this" correctement dans la fonction handleSubmit
    this.handleSubmit = this.handleSubmit.bind(this);
  }
 // Définition de la méthode "handleSubmit"
 // Elle sera appelée lorsque l'utilisateur soumettra le formulaire
  handleSubmit(event) {
    event.preventDefault(); //Empêche le rechargement automatique de la page (comportement par défaut d'un formulaire)
    alert("Formulaire soumis !"); // Affiche une boîte de message lorsque le formulaire est envoyé
  }
  }
// La méthode render() décrit ce que le composant doit afficher à l'écran
  render() {
```

```
  return (
// Le formulaire HTML
  // Quand on le soumet, la fonction handleSubmit est appelée
    <form onSubmit={this.handleSubmit}>
  {/* Le bouton de type "submit" permet d'envoyer le formulaire */}
      <button type="submit">Envoyer</button>
    </form>
  );
 }
}
// On exporte le composant "Formulaire" pour pouvoir l'utiliser dans d'autres fichiers
export default Formulaire;
```

onChange

```
// On importe React et la classe "Component" depuis la bibliothèque "react"
import React, { Component } from "react";
// On crée une nouvelle classe "FormulaireNom" qui hérite de "Component"
class FormulaireNom extends Component {
 // Le constructeur est appelé automatiquement lors de la création du composant
 constructor() {
   super();// On appelle le constructeur parent (obligatoire)
 // On initialise l'état (state) du composant avec une propriété "nom" vide
  // Le state permet de stocker et de gérer les données internes du composant
```

```
  this.state = { nom: "" };
 }
 // Méthode "changerNom" : appelée chaque fois que l'utilisateur tape quelque chose dans le champ texte
// "event" contient les informations sur ce qui s'est passé (la frappe clavier ici)
 changerNom = (event) => {
 // On met à jour la propriété "nom" du state avec la nouvelle valeur saisie
  this.setState({ nom: event.target.value });
 };
// La méthode render() décrit ce que le composant doit afficher à  l'écran
 render() {
  return (
    <div>
{/* Une étiquette (label) pour indiquer le champ à remplir */}
      <label>Nom :</label>
{/* Champ de texte :
      - type="text" indique qu'on saisit du texte
      - onChange={this.changerNom} indique que chaque modification déclenche la fonction changerNom */}
      <input type="text" onChange={this.changerNom} />
{/* Affiche un message de bienvenue en utilisant la valeur actuelle du state.nom */}
      <p>Bonjour {this.state.nom}</p>
    </div>
  );
 }
```

```
}
// On exporte le composant pour pouvoir l'utiliser ailleurs
export default FormulaireNom;
```

Exemples pratiques

Exemple 1 – Formulaire d'inscription

```
// On importe React et la classe "Component" depuis la bibliothèque "react"
import React, { Component } from "react";
// On crée une classe "Inscription" qui hérite de "Component"
class Inscription extends Component {
// Le constructeur s'exécute une seule fois à la création du composant
  constructor() {
    super();// Appel du constructeur parent (obligatoire)
// On initialise le "state" (l'état du composant)
   // Il contient 3 propriétés :
   // - nom : le nom saisi par l'utilisateur
   // - prenom : le prénom saisi par l'utilisateur
   // - info : le texte final qui affichera les deux informations
    this.state = {
      nom: "",
      prenom: "",
      info: "",
```

```
  };
}
// Méthode appelée quand l'utilisateur tape dans le champ "Nom"
changerNom = (e) => {
// On met à jour la valeur de "nom" dans le state
// e.target.value : le texte saisi
// toUpperCase() : transforme le texte en majuscules
  this.setState({ nom: e.target.value.toUpperCase() });
};
// Méthode appelée quand l'utilisateur tape dans le champ "Prénom"
changerPrenom = (e) => {
// Même principe que changerNom, mais pour le prénom
  this.setState({ prenom: e.target.value.toUpperCase() });
};
// Méthode appelée quand on clique sur le bouton "Afficher"
envoyer = () => {
  // On met à jour la propriété "info" du state
// Elle contient une phrase combinant le nom et le prénom saisis
this.setState({
    info: `Nom : ${this.state.nom} | Prénom : ${this.state.prenom}`,
  });
};
// La méthode render() décrit le contenu à afficher à l'écran
```

```
  render() {
    return (
      <div>
 {/* Champ de saisie pour le nom */}
        <label>Nom :</label>
        <input type="text" onChange={this.changerNom} />
 {/* Champ de saisie pour le prénom */}
        <label>Prénom :</label>
        <input type="text" onChange={this.changerPrenom} />
    {/* Bouton qui exécute la fonction envoyer() au clic */}
        <button onClick={this.envoyer}>Afficher</button>
{/* Paragraphe qui affiche les informations enregistrées dans "info" */}
        <p>{this.state.info}</p>
      </div>
    );
  }
}
// On exporte le composant pour pouvoir l'utiliser dans un autre fichier
export default Inscription;
```

Explication :

- **Chaque input met à jour une partie du state.**
- **this.setState() met à jour le contenu du composant.**

- **this.state.info affiche la concaténation après le clic.**

Exemple 2 – Validation d'un mot de passe

```
// On importe React et la classe "Component" depuis la bibliothèque "react"
import React, { Component } from "react";
// On crée une classe "PasswordValidator" qui hérite de "Component"
class PasswordValidator extends Component {
 // Le constructeur est appelé automatiquement à la création du composant
  constructor() {
    super();// Appel du constructeur parent (obligatoire)
 // On initialise le state avec une seule propriété :
  // "password" représente le texte saisi dans le champ mot de passe
    this.state = { password: "" };
  }
// Méthode appelée à chaque fois que l'utilisateur modifie le champ de mot de passe
  handleChange = (e) => {
// On met à jour la valeur du state avec le nouveau texte saisi
  // e.target.value contient la valeur actuelle du champ
    this.setState({ password: e.target.value });
  };
 // La méthode render() décrit ce que le composant doit afficher
  render() {
```

```
return (
  <div>
{/* Étiquette du champ mot de passe */}
    <label>Mot de passe :</label>
  {/* Champ de saisie pour le mot de passe */}
    <input
      type="password"  // type "password" cache les caractères saisis
      value={this.state.password}   // la valeur du champ vient du  state
      onChange={this.handleChange}  // chaque frappe clavier appelle handleChange
    />
{/*
    Affichage conditionnel :
    Si le mot de passe n'est pas vide (longueur > 0)
    ET qu'il contient moins de 4 caractères,
    alors on affiche un message d'erreur en rouge.
  */}
{this.state.password.length > 0 && this.state.password.length < 4 && (
      <p style={{ color: "red" }}>
        Le mot de passe doit avoir au moins 4 caractères
      </p>
    )}
  </div>
```

```
  );
 }
}
// On exporte le composant pour pouvoir l'utiliser ailleurs
export default PasswordValidator;
```

Explication :

- **Le state stocke la valeur du mot de passe.**
- **À chaque frappe (onChange), this.setState() met à jour.**
- **Une condition affiche un message si le mot de passe est trop court.**

Communication entre composants (Parent ↔ Enfant)

Principe général

Type de communication	Sens	Moyen
Parent → Enfant	Envoi de données	via props
Enfant → Parent	Envoi d'une action	via une fonction passée en props

Exemple : Recherche de fruits et légumes

App.js

```
// On importe React et la classe Component depuis la bibliothèque "react"
import React, { Component } from "react";
// On importe les deux composants enfants
import ChercheBar from "./ChercheBar";
import ResultatList from "./ResultatList";
// Définition du composant principal App
class App extends Component {
  constructor() {
    super();// Appel du constructeur parent
// Initialisation du state (données internes du composant)
    this.state = {
      type: "", // Type recherché (ex: FRUIT ou LEGUME)
      resultats: [], // Tableau des résultats à afficher
    };
// On lie la méthode "onChercheSubmitApp" à l'objet courant (this)
    this.onChercheSubmitApp = this.onChercheSubmitApp.bind(this);
  }
// Fonction appelée quand le formulaire de recherche est soumis depuis le composant enfant ChercheBar
  onChercheSubmitApp(typeRecherche) {
  // Liste fixe de fruits et légumes
    const list = [
```

```
    { nom: "banane", type: "fruit" },
    { nom: "orange", type: "fruit" },
    { nom: "tomate", type: "legume" },
    { nom: "carotte", type: "legume" },
  ];
// On filtre la liste selon le type saisi (fruit ou légume)
 // .filter() crée un nouveau tableau contenant seulement les éléments correspondant au type
  const resultatFiltre = list.filter(
    (item) => item.type.toUpperCase() === typeRecherche
  );
 // On met à jour le state : type recherché et résultats trouvés
  this.setState({
    type: typeRecherche,
    resultats: resultatFiltre,
  });
 }
// La méthode render() affiche le contenu du composant
 render() {
  return (
    <div>
      <h1>Composant App</h1>
 {/* On affiche le composant ChercheBar et on lui passe une fonction en prop */}
```

```
    {/* Cette fonction sera appelée quand l'utilisateur soumettra le formulaire */}
      <ChercheBar onChercheSubmit={this.onChercheSubmitApp} />
    {/* On affiche le type recherché */}
      <p>Type recherché : {this.state.type}</p>
 {/* On affiche le composant ResultatList avec la liste des résultats */}
      <ResultatList resultats={this.state.resultats} />
     </div>
   );
  }
}
// On exporte le composant App pour pouvoir l'utiliser dans index.js
export default App;
```

ChercheBar.js

```
// On importe React et Component depuis "react"
import React, { Component } from "react";
// Définition du composant ChercheBar
class ChercheBar extends Component {
  constructor() {
    super();
// Le state contient un champ "term" qui garde le texte tapé par l'utilisateur
    this.state = { term: "" };
  }
```

```
// Fonction appelée quand le formulaire est soumis
 handleSubmit = (e) => {
   e.preventDefault();// Empêche le rechargement de la page
// On appelle la fonction transmise par le parent (App.js)
 // et on lui envoie la valeur saisie (term)
   this.props.onChercheSubmit(this.state.term);
 };
// Fonction appelée à chaque fois que l'utilisateur tape quelque chose dans le champ
 handleChange = (e) => {
 // On met à jour le state avec la valeur tapée, convertie en majuscules
   this.setState({ term: e.target.value.toUpperCase() });
 };
// La méthode render() affiche le formulaire
 render() {
   return (
     <form onSubmit={this.handleSubmit}>
       <label>Entrer le type :</label>
       <input
         type="text"
         value={this.state.term}  // Sa valeur vient du state
         onChange={this.handleChange} // Chaque modification appelle handleChange
       />
```

```
      <button type="submit">Chercher</button>
    </form>
  );
 }
}
// On exporte le composant
export default ChercheBar;
```

ResultatList.js

```
// On importe React et Component depuis "react"
import React, { Component } from "react";
// Définition du composant ResultatList
class ResultatList extends Component {
 render() {
  // On récupère la liste des résultats depuis les props envoyées par le parent (App)
   const { resultats } = this.props;
   return (
    <div>
      <h2>Résultats</h2>
     {/* Si aucun résultat, on affiche un message "Pas de résultats" */}
      {resultats.length === 0 ? (
        <p>Pas de résultats</p>
        ) : (
```

```
// Sinon, on affiche la liste des éléments trouvés
        <ul>
{/* On parcourt le tableau des résultats avec map() */}
      {/* Pour chaque élément, on crée une balise <li> affichant le nom */}
          {resultats.map((item) => (
            <li key={item.nom}>{item.nom}</li>
          ))}
        </ul>
      )}
    </div>
  );
 }
}
// On exporte le composant
export default ResultatList;
```

<u>Explication du flux :</u>

1. **App envoie une fonction onChercheSubmitApp() à ChercheBar.**
2. **L'enfant (ChercheBar) appelle cette fonction avec la valeur saisie.**
3. **Le parent (App) met à jour son state → filtre la liste.**
4. **Les résultats sont transmis à ResultatList via props.**

Résumé global

Élément	Rôle	Exemple
state	Données internes du composant	this.state = { nom: "" }
setState()	Met à jour le state	this.setState({ nom: "Ali" })
props	Données reçues d'un parent	this.props.nom
onClick / onChange / onSubmit	Gestionnaires d'événements	<button onClick={...}>

OFPPT

Formateur : AARAB HAFID